
ASSESSING OPENEVOLVE AS AN EVOLUTIONARY CODING FRAMEWORK FOR UNCORRELATED TRACK PROCESSING ALGORITHM DEVELOPMENT

KYLE F. GALANG, AURELA BROQI, AARON NOGUES, RUBEN DENNIS, EZRA STONE

1 Executive Summary

Identifying unknown objects in space from sparse, noisy, and time-sensitive observations remains a significant challenge in Space Domain Awareness (SDA). Accurate orbit estimation is critical for detecting, correlating, and responding to potential threats. However, existing development approaches are often manual, time-intensive, and difficult to adapt with the usage of modern adversarial techniques such as camouflage, concealment, deception, and maneuvering that degrade tracking reliability. A key limitation in Uncorrelated Track Processing (UCTP) is the absence of established baseline solutions which restrict effective benchmarking, evaluation, and systematic algorithm improvement over time.

This research investigates the viability of OpenEvolve, an open-source evolutionary coding agent that leverages large language models to iteratively optimize code, as a tool for supporting UCTP algorithm development and benchmarking. The study evaluates OpenEvolve's ability to automate elements of algorithm discovery, testing, and refinement while integrating it with existing UCT performance metrics and evaluation frameworks.

To support this effort, an analytics console will be developed in parallel with the research to bridge both OpenEvolve and UCTP implementation. The analytics console will provide real-time visibility into program evolution and performance. This tool will enhance transparency and reproducibility efforts by enabling researchers to monitor optimization trends, identify performance peaks and degradation, and better help understand how algorithms evolve over time in OpenEvolve. Together, this work aims to establish a scalable and data-driven approach to improving UCTP methodologies.

Duration: 8 months.

2 Goals and Impact

The primary goal of this research is to evaluate the viability of OpenEvolve as a tool for automating algorithm development and benchmarking in Uncorrelated Track Processing (UCTP). Specifically, this project seeks to determine whether evolutionary coding agents can accelerate algorithm discovery, testing, and refinement while reducing reliance on manual development workflows. Another key goal is to integrate OpenEvolve with existing UCT performance metrics and evaluation frameworks to support systematic and repeatable benchmarking.

A parallel objective is the development of an analytics console that provides real-time visibility into program evolution and performance. This tool is intended to improve transparency and reproducibility efforts by allowing researchers to monitor optimization progress, identify performance trends, and evaluate where algorithm performance improves or declines over time.

The expected impact of this work is the creation of a scalable, data-driven framework for UCTP development and evaluation. By automating parts of the algorithm design process and improving benchmarking capability, this research could reduce development time, improve solution quality, and provide a stronger foundation for future UCTP research. This effort will be done through a series of experimentations to assess the viability of OpenEvolve.

3 Technical Plan

The technical execution of this research and development effort is organized into two primary phases. Due to project time constraints and available manpower, these phases will be conducted concurrently rather than sequentially. The principal research deliverable to the sponsors is a formal assessment of OpenEvolve’s viability as a framework for Uncorrelated Track Processing (UCTP) benchmarking and algorithm development. How these phases will be conducted:

Phase I will focus on establishing and validating OpenEvolve environments for proof-of-concept experimentation. This phase will include the development of simplified UCTP logic and representative initial programs that can be introduced into OpenEvolve for controlled evaluation. The purpose of this phase is to determine whether the framework can effectively support smaller-scale UCTP-related tasks before being applied to more operationally relevant benchmarking processes.

Phase II will focus on integrating existing UCTP benchmarking methodologies and performance evaluation metrics into the OpenEvolve framework. This phase will examine which components should be used from the current UCTP workflows alongside assessing whether OpenEvolve can generate meaningful algorithmic improvements. In addition, Phase II will support the continued refinement of the simplified logic developed during Phase I, enabling its progression toward a more representative processor used in real life.

Concurrently with the research effort, an analytics console will be designed and implemented to provide real-time visibility into program evolution and system performance. This console will allow researchers to observe optimization behavior throughout the experimental lifecycle, including the identification of performance peaks, degradation trends, and the effects of code evolution over time. The development of this tool is intended to assist researchers in identifying how OpenEvolve performs during the experiments.

3.1 Phase I: OpenEvolve Environment and UCTP Development

3.1.1 OpenEvolve Setup

OpenEvolve will be installed on each researcher’s local machine to support hands on evaluation of the framework and its architecture. This setup will enable researchers to develop a practical understanding of OpenEvolve’s three required components: (1) the initial program file, (2) the evaluator file, and (3) the configuration file. Researchers will begin by executing existing proof-of-concept example problems provided by OpenEvolve and will then modify these examples in a controlled manner to observe how the evolutionary process responds to changes in the evaluation file and configuration settings. This phase is

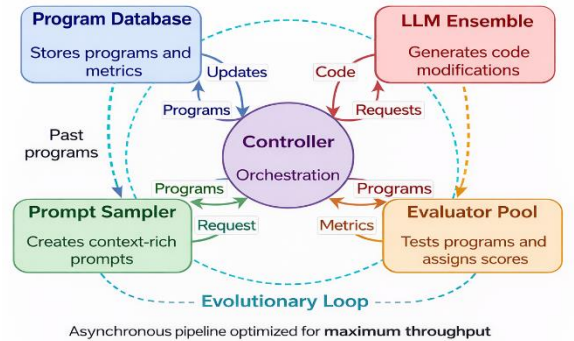


Figure 1: OpenEvolve Architecture

intended to establish familiarity with OpenEvolve’s operational workflow and testing to see how OpenEvolve performance can be changed through prompting. It is important to note that many researchers can have different ways of prompting to solve the problems, these tests check to see if evolution is hard bounded by the metrics or prompting techniques.

3.2 Phase II: Domain-Specific Integration

3.2.1 Scaled-down UCTP Development

As researchers become more proficient in utilizing OpenEvolve’s architecture, this phase will focus on developing initial UCTP programs that align with the benchmarking workflow established by the previous team. This includes designing programs that ingest data generated by the benchmarking framework and produce outputs that conform to the expected UCTP output schemas. To evaluate OpenEvolve across varying levels of problem fidelity and complexity, the development of scaled-down UCTP initial programs will be organized into three separate environments. This structure will support the assessment of OpenEvolve’s performance under different configurations, while also providing corresponding cost estimates for each level of experimentation.

4 Technical

4.1 Phase I: OpenEvolve Testing and Cost Evaluation

Phase I focused on configuring OpenEvolve environments on researchers’ local machines and executing proof-of-concept problems to validate system functionality, including the setup and usage of API keys through Google AI Studio’s free-tier access.

LLM	Per Iteration Estimate		Iterations						
	Min	Max	10	15	30	50	75	200	500
o3	\$0.15	\$0.60	\$1.50 - \$6.00	\$2.25 - \$9.00	\$4.50 - \$18.00	\$7.50 - \$30.00	\$11.25 - \$45.00	\$30.00 - \$120.00	\$75.00 - \$300.00
o3-mini	\$0.03	\$0.12	\$0.30 - \$1.20	\$0.45 - \$1.80	\$0.90 - \$3.60	\$1.50 - \$6.00	\$2.25 - \$9.00	\$6.00 - \$24.00	\$15.00 - \$60.00
Gemini 2.5 Pro	\$0.08	\$0.30	\$0.80 - \$3.00	\$1.20 - \$4.50	\$2.40 - \$9.00	\$4.00 - \$15.00	\$6.00 - \$22.50	\$16.00 - \$60.00	\$40.00 - \$150.00
Gemini 2.5 Flash	\$0.01	\$0.05	\$0.10 - \$0.50	\$0.15 - \$0.75	\$0.30 - \$1.50	\$0.50 - \$2.50	\$0.75 - \$3.75	\$2.00 - \$10.00	\$5.00 - \$25.00
Local Models	\$0.00	\$0.01	\$0.00 - \$0.10	\$0.00 - \$0.15	\$0.00 - \$0.30	\$0.00 - \$0.50	\$0.00 - \$0.75	\$0.00 - \$2.00	\$0.00 - \$5.00

Figure 2: OpenEvolve Price Estimations per Model

Referring to Figure 2, OpenEvolve cost estimates are presented as minimum and maximum ranges to account for the framework’s non-deterministic execution process. Although higher-capability models are associated with greater operational cost, proof-of-concept experiments indicated that Gemini 2.5 Flash offered the most favorable balance between performance and cost efficiency and was therefore selected as the baseline model for evaluation. These cost estimates were considered when setting up the three separate experimental environments with increasing levels of fidelity relative to Uncorrelated Track Processing (UCTP).

4.2 Phase II: Scaled-down UCTP Development Environments

The first environment evaluated proof-of-concept problems provided by OpenEvolve that were conceptually related to UCTP, such as symbolic regression and circle packing. The second environment tested initial processor logic developed by researchers. The third environment evaluated a more refined implementation for grouping object tracks under additional filtering constraints.

4.2.1 Phase II: Experiment No.1: Proof-of-Concepts Environment

The first experiment showed that when running OpenEvolve example problems conceptually related to UCTP, program evaluation metrics often plateaued early in the optimization process. This behavior was not readily apparent until the Analytics Console was developed, which enabled researchers to monitor program performance in real time through line graphs displaying iteration count on the x-axis and metric scores on the y-axis. This visualization made it possible to identify when performance improvements began to level off and provided greater insight into the evolution of candidate programs over time.

4.2.2 Phase II: Experiment No.2: Scaled-down UCTP Gating Logic

After gaining familiarity with developing custom UCTP programs within OpenEvolve, the second experiment focused on evaluating the trade-offs in performance metrics associated with the fidelity of the initial program provided by the researchers. In this phase, researchers have already exhausted the free-tier capacity available through Google AI Studio. As a result, initial financial investment was required to continue experimentation and to assess the cost estimates associated with developing and testing custom problem sets in OpenEvolve.

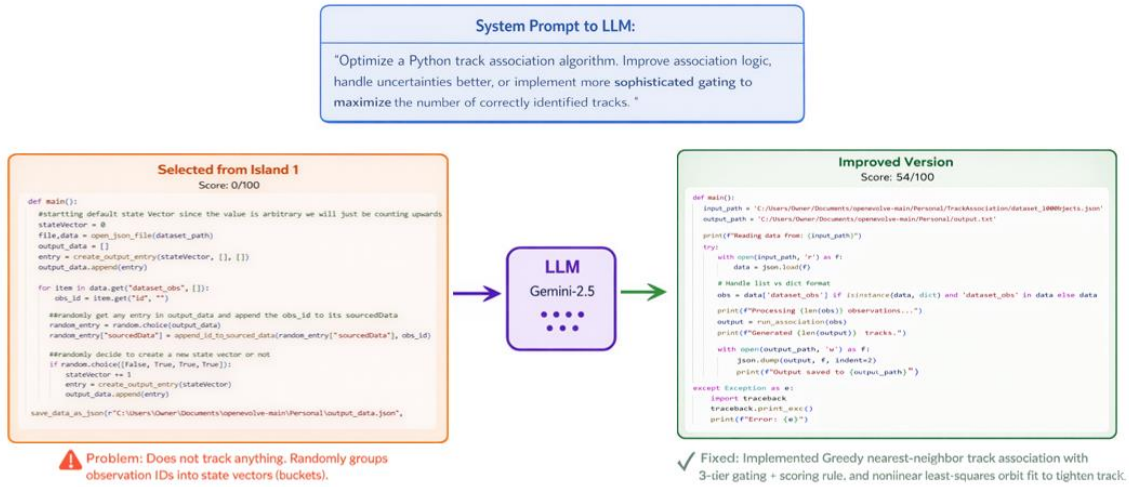


Figure 3: Experiment No.2: Simple Grouping Logic Experiment

The first program developed generates a randomized output file from an input observation dataset of 10 objects. It begins by loading the JSON dataset and defines an output structure for candidate state vectors that conforms to the UCTP benchmarking framework output schema, including fields such as idStateVector, sourcedData, placeholder position and velocity values, and other required metadata. The program then iterates through each observation in dataset_obs and randomly assigns each observation ID to one of the existing candidate state vectors by appending it to that entry's sourcedData list. At the same time, it randomly decides whether to create a new state vector, causing the number of output entries to grow over time. The result is a synthetic output file in which observations are grouped arbitrarily rather than through any physical tracking or association logic. This program is best understood as a baseline or test generator for producing placeholder association outputs, rather than as a true UCTP algorithm to test how an initial program can evolve from the metrics the user defines in OpenEvolve.

The program is evaluated by comparing its randomly generated observation groupings against the ground-truth reference groupings in the dataset. The evaluation file computes an F1 score between

each reference track and the best matching output track based on shared observation IDs in sourcedData, and the final score is the average of these best-match F1 values. As a result, the program is judged on how closely its random track assignments resemble the true grouping of observations, rather than on physical orbit quality or state estimation accuracy.

This experiment was designed to demonstrate how effectively OpenEvolve can improve a simple initial program related to UCTP toward the desired evaluation metrics. As shown in Figure 3, the initial program scored 0 out of 100 prior to any evolution. This result was expected, as the program did not contain meaningful association logic and instead grouped observations randomly, making it unable to track objects effectively. Based on the evaluation metrics defined by the researchers, the objective of this experiment was to guide OpenEvolve toward developing logic that is more closely aligned with the scoring criteria and improved overall track association performance.

After the evolutionary runs, this experiment concluded that OpenEvolve is effective at generating useful logic from very simple initial programs. Using only one island and 30 iterations, OpenEvolve was able to improve the program's score from 0/100 to 54/100 against the same evaluation file. This was a significant result because it demonstrated that meaningful track-association logic could emerge even when starting from a minimal baseline. This finding was especially important in the context of the UCTP benchmarking framework, where the dataset inputs and evaluation metrics had already been defined by the previous team, but the core processor logic itself was still absent. As a result, the next objective was to determine whether OpenEvolve could help reconstruct a functional processor by leveraging the available input data generated by the benchmarking framework and the scoring criteria derived from existing processor outputs.

The second program was developed concurrently with experiment No. 1. This program focused on improving object observability and track continuity. In the first stage, the program takes in the time and directional measurements of observed objects and sorts them in order. This step is important because telescopes and sensors have only a limited observation window before the object moves out of view, at which point continued tracking must be handed off to another sensor located elsewhere. In the second stage, the program attempts to merge these short observation tracklets and stitch them together into a single continuous track, as illustrated in Figure 4. The primary purpose of this program was to evaluate OpenEvolve on a more complex initial problem and determine how much additional performance and logic refinement could be achieved through its evolutionary process.

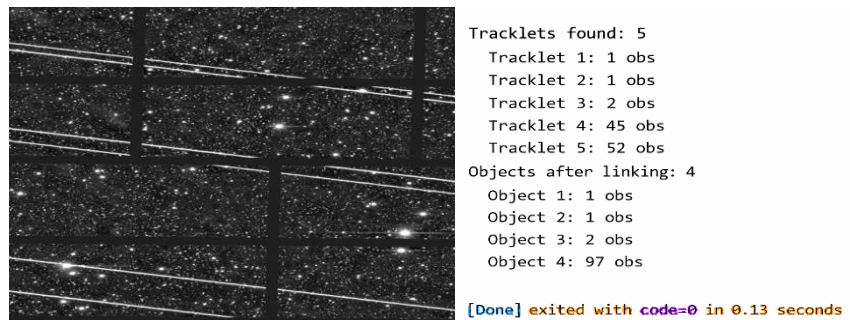


Figure 4: Clustering observations into short tracks and stitching them together to form bigger tracks across the sky.

One of the primary challenges in these experiments was isolating the logic to ensure that OpenEvolve's optimization process did not become overly convoluted. For this reason, the programs developed during the experimentation focused on specific forms of logic used within UCT processors, rather than attempting to replicate an entire processor and the many problems it intends to solve. This was a significant concern for the researchers because available token resources were limited during experimentation. As a result, the objective was to determine how efficiently OpenEvolve could optimize initial programs and develop useful logic based on evaluation metrics defined by the researchers with as little iterations and token usage as possible. It is also important to note that, as development progressed, the research shifted away from custom-designed metrics and towards existing metrics already used to evaluate real processor performance within the UCTP benchmarking framework.

These metrics are the state metrics which propagate each reference to candidates and compares state vectors, residual metrics that computes the unit circle great arc residual between two sets of angular observations which then measures the accuracy and precision of the UCTP, and binary metrics which calculate the true-false positives and true-false negatives in the observations.

As discussed before, the problem gets more difficult as more logic gets implemented into the initial and evaluator programs. Where if we try to base the efficiency and how well our program performs strictly based off of the combined score referenced from figure 5. How does OpenEvolve improve that score, and what trade-offs are made in the process? This question was explored through the experimentation conducted with this program. As shown in Figures 6 and 7, improvements in residual score were accompanied by declines in binary score. These results raised an important concern for researchers. What does it actually mean for a program to perform better when improvement in one metric may come at the expense of another? Alongside what if that improvement weighs more heavily than the degradation of other metrics which causes the combined score to increase although other metrics are performing worse.

This segways to the final experimentations of this research performed in phase 2. Where researchers tested what it actually means when a program performs better based off of the evaluator files developed by users using OpenEvolve.

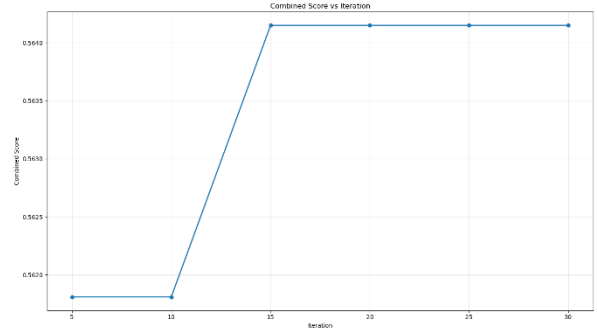


Figure 5: Combined Score vs. Iteration

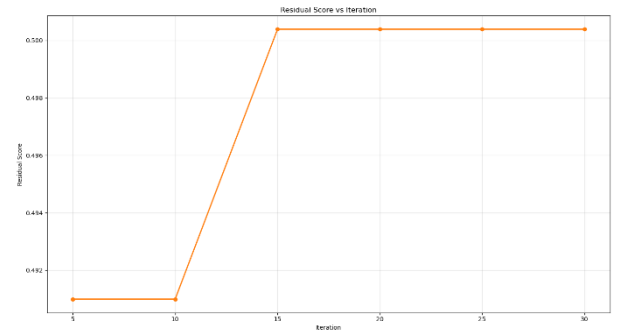


Figure 6: Residual Score vs. Iteration

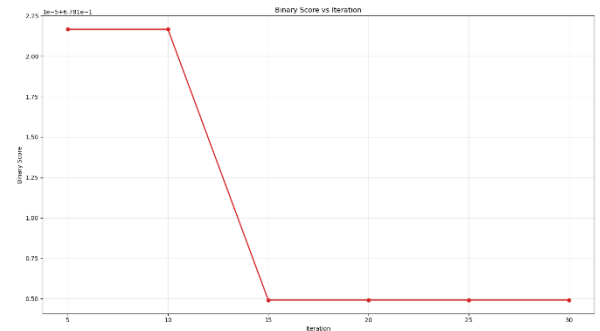


Figure 7: Binary Score vs. Iteration

4.2.3 Phase II: Experiment No.3: Refined UCTP Gating Logic with Filtering

This final experiment builds upon all previously developed initial programs and aims to analyze the trade-offs between using simpler initial programs versus more complex initial programs within OpenEvolve. Specifically, it evaluates the performance differences observed when evolving higher-fidelity programs and examines how increased complexity impacts optimization outcomes. Additionally, this experiment seeks to clarify what it means for a program to perform “better,” particularly in cases where improvements in one evaluation metric may come at the expense of others. This experiment is set up where researchers developed two initial programs at a higher level of fidelity correlating to UCTP logic. These two initial programs are:

1. Base Program:

The base initial program implements a deterministic prototype for Uncorrelated Track Processing by converting raw EO observations into candidate object tracks and state vectors. It begins by loading observation data, transforming each observation’s right ascension and declination into a unit direction vector, and converting sensor positions from geodetic coordinates into the J2000 inertial frame using ECEF-to-ECI rotation based on Greenwich Mean Sidereal Time. Using each sensor position, range measurement, and direction vector, the program estimates object positions in inertial space. It then sorts observations by time and applies a Kalman-filter-style association process, where existing tracks are predicted forward using a constant-velocity motion model and compared against new observations through a Mahalanobis-distance gate controlled by tunable parameters such as process noise and gating threshold. Observations that meet the gating criteria are added to existing tracks and used to update the track state; otherwise, new tracks are created. The final result is written as an output file containing grouped observation IDs and estimated position and velocity state vectors for each candidate track.

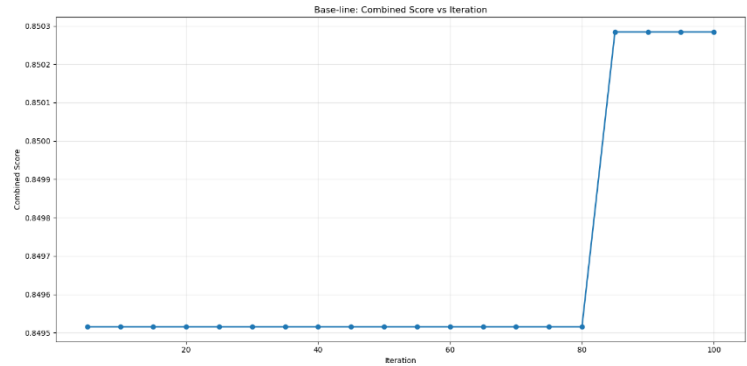


Figure 8: Base Program Combined Score vs Iteration

2. Loophole Program:

This loophole initial program is a more opportunistic version of the base initial program because it is designed to exploit weaknesses in the evaluation setup rather than focus only on physically consistent orbit tracking. Like the previous program, it still converts EO observations into J2000 positions, uses uncertainty-aware gating, and produces UCTP-style grouped tracks and state vectors. However, it adds several behaviors that make it a loophole: it

first detects whether the input resembles an orbit problem, a 2D Cartesian problem, or a 1D angle problem, and then switches into whichever pipeline is most convenient for scoring; it introduces special case logic for circle-style datasets and angle clustering that are unrelated to true UCTP

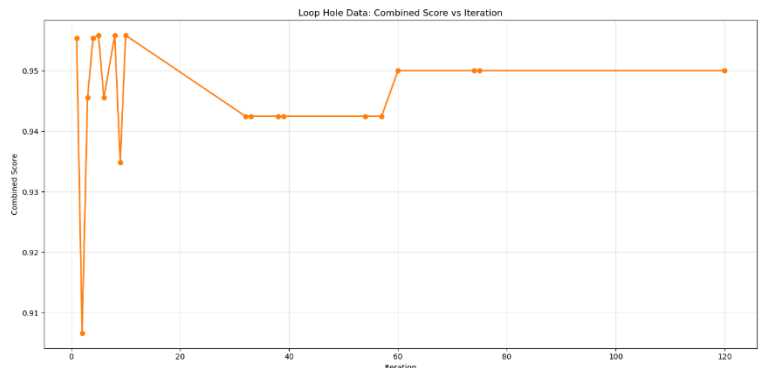


Figure 9: Loophole Program Combined Score vs Iteration

processing; and most importantly it uses a Monte Carlo Markov Chain style proposal loop with track reassign, split, and merge moves to search for track groupings that maximize the scoring objective. Its hypothesis score also explicitly rewards longer tracks and penalizes fragmentation, which helps it exploit the evaluator by optimizing grouping behavior directly. Compared with the previous deterministic Kalman-gating program, this version is considered a loophole because it broadens the problem definition, uses dataset-type branching and stochastic hypothesis search, and exploits the evaluator’s grouping-based rewards to improve score even when the added logic does not necessarily reflect a more realistic or operationally faithful UCTP solution.

The main idea of these two programs is to see how well we can fit our initial programs to our desired evaluator metrics. Where both programs have a higher level of fidelity in logic corresponding to UCTP. We have a base function that does normal evolutions and a loophole function that tries to exploit evaluation metrics to test what it really means to have an improved program based off OpenEvolve.

Both initial programs were run with similar evaluator files where it’s responsible for executing candidate programs and quantitatively assessing their performance against a ground-truth dataset. It runs the candidate program, loads its generated output, and compares the predicted observation groupings to the reference groupings defined in the dataset. For tracking problems, the evaluation is performed through pairwise comparison of observation IDs, computing true positives, false positives, false negatives, and true negatives to derive metrics such as precision, recall, specificity, F1 score, and a weighted combined score. The combined score is further adjusted through soft penalties and rewards, including penalties for excessive false positives and minimal track counts, as well as small rewards for producing a reasonable number of tracks. Because the evaluation emphasizes grouping accuracy and includes heuristic adjustments to the final score, it creates opportunities for programs to optimize specifically toward these metrics rather than strictly improving physical tracking accuracy.

4.3 Experimentation Conclusion

This series of experiments demonstrates that OpenEvolve is capable of generating and improving algorithmic logic from minimal initial programs, particularly when structured datasets and clearly defined evaluation metrics are available. Early results showed that even a random baseline program could evolve into a partially functional track-association solution, highlighting OpenEvolve’s ability to discover meaningful logic through iterative optimization. However, as program complexity increased, the experiments revealed important trade-offs in how candidate programs were scored by the evaluator files developed by the researchers.

A key finding of this research is that performance within OpenEvolve is highly dependent on evaluator design. This means that users must be careful when constructing evaluator files, as the evaluation criteria directly shape how the evolutionary process defines “improvement.” As shown in Figure 8, one program exhibited a more linear improvement trend over time. In contrast, Figure 9 reflects a more sophisticated initial program and evaluator structure, where penalties and rewards were introduced to encourage the system to balance multiple user-defined metrics. Building on the results of Experiment No. 2, the researchers examined how much logic should be distributed between the initial and evaluator programs. This question carried into Experiment No. 3, where two different development approaches were tested: one emphasizing more direct linear improvement, and another emphasizing optimization trade-offs through evaluator-driven rewards and penalties towards the initial program during evolutions. This represented an important breakthrough, as the earlier scaled-down UCTP programs were comparatively simple. As evaluator

complexity increased, the results provided a more representative view of how OpenEvolve handles complex problems over time. Rather than producing only steady linear gains, the system began seeking more stable solutions that accounted for multiple competing metrics defined by the evaluator.

Additionally, these experiments highlight the importance of balancing simplicity and fidelity in initial program design. Simpler programs enable OpenEvolve to explore the solution space more efficiently and discover foundational logic, whereas more complex programs provide stronger starting points but introduce greater optimization difficulty and computational cost. Resource limitations were also evaluated more particularly in token usage and computing expense. This further reinforces the need for careful problem decomposition and targeted logic development when applying OpenEvolve to more sophisticated UCTP-related tasks. Rather than trying to optimize larger-scale frameworks with many moving parts.

Additional to these experiments, cost evaluations were conducted throughout all the experiments. Although researchers were conducting these experiments on local machines with very limited tokens. The cost metrics were heavily monitored to showcase to sponsors the cost of using OpenEvolve on a larger scale.

Figure 10 presents the total input token usage across the three experimental environments. Experiment No. 1 corresponds to proof-of-concept runs, Experiment No. 2 to scaled-down UCTP logic, and Experiment No. 3 to refined UCTP logic. As shown in the figure, Experiment No. 3 required more tokens than both Experiments No. 1 and No. 2. This indicates that, within OpenEvolve, greater problem complexity is closely associated with increased computational cost.

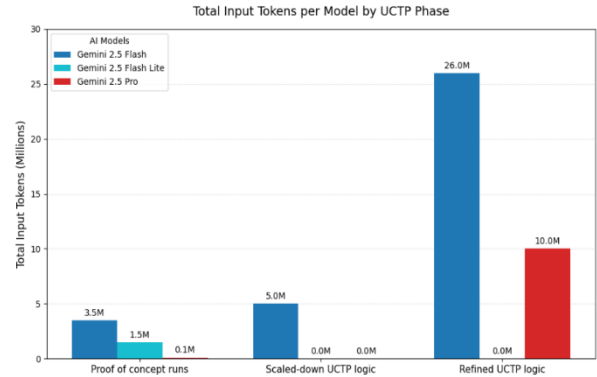


Figure 10: Total Input Tokens in Experimentation

Figure 11 presents the total API requests and corresponding success rates for each experiment. Experiment No. 3 produced a much higher number of API requests but achieved a lower success rate, reflecting the added complexity of its initial and evaluator programs. From these results, the researchers concluded that OpenEvolve is more efficient when complex problems are decomposed into smaller, more targeted optimization tasks. Under this approach, experiments tend to achieve higher success rates and lower overall cost than runs that attempt to optimize highly complex programs with many interacting forms of logic.

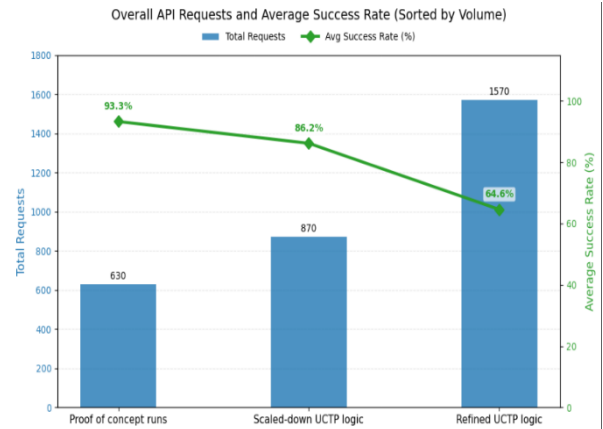


Figure 11: Overall API Requests and Success Rate

4.4 Analytic Console Development

Working with OpenEvolve initially made it difficult to determine when evolving programs began to plateau in performance. To address this challenge, the analytics console was developed in parallel with the research effort. The console provides real-time visibility into program evolution and performance, allowing researchers to identify when performance peaks, when it declines, and how the optimization process changes over time with greater transparency and reproducibility. This significantly improved the research workflow, as performance graphs were generated automatically rather than requiring users to manually monitor OpenEvolve status through command-line outputs.

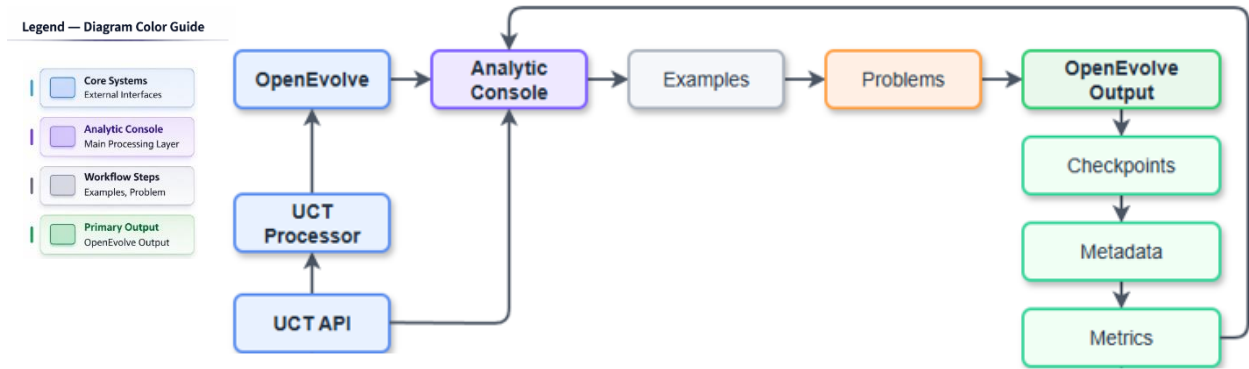


Figure 12: Analytic Console Data Flow

The analytics console operates by ingesting raw observation data from the Unified Data Library through the UCT API and supplying that data both to the processor and to the console itself, ensuring that researchers can clearly track the data being used during experimentation. OpenEvolve is then used to evolve the processor, and the resulting outputs are compared against real processor outputs for evaluation. Real-time evolution updates are obtained by continuously pinging the file system repository that stores the selected best programs throughout the evolutionary process. From these stored checkpoint files, metric data in JSON format is collected after each iteration and transmitted to the analytics console, enabling researchers to visualize performance changes as evolution progresses.

5 Conclusion

OpenEvolve presents a viable optimization tool for isolated logic and other programs. The framework demonstrates the ability to improve code across the initial proof-of-concept problem categories, its operational utility is presently constrained by prolonged iteration times, extensive setup requirements, and a heavy reliance on token expenditures. In its current form, OpenEvolve can successfully optimize isolated logic and scaled-down algorithms. However, applying it to problems like UCTP which require a large knowledge base of the subject introduces significant logistical and financial overhead. The evolutionary architecture currently exhibits difficulty spanning complex problem spaces within a reasonable number of iterations. This limitation directly inflates API token usage and impacts the overall cost-efficiency, remaining a primary area of concern for future deployment and research.

References

- [1] Sharma, A. (2025). OpenEvolve: An open-source evolutionary coding agent [Software]. GitHub. <https://github.com/algorithmicsuperintelligence/openevolve>
- [2] Shojaee, P., Nguyen, N.-H., Meidani, K., Farimani, A. B., Doan, K. D., & Reddy, C. K. (2025). LLM-SRBench: A new benchmark for scientific equation discovery with large language models. arXiv.
- [3] Caves, L., Puchner, M., Stivi, B., Lundin, G., & Smith, C. (n.d.). Benchmarking and evaluating uncorrelated track processing: Common task framework at the SDA TAP Lab.
- [4] Oh, S., Russell, S., & Sastry, S. (2005). Markov chain Monte Carlo data association for multiple-target tracking.
- [5] Press, O., Amos, B., Zhao, H., Wu, Y., Ainsworth, S. K., Krupke, D., Kidger, P., Sajed, T., Stellato, B., Park, J., Bosch, N., Meril, E., Steppi, A., Zharmagambetov, A., Zhang, F., Perez-Pineiro, D., Mercurio, A., Zhan, N., Abramovich, T., Lieret, K., Zhang, H., Huang, S., Bethge, M., & Press, O. (2024). AlgoTune: Can language models speed up general-purpose numerical programs?